
Curso de especialización en Desarrollo de aplicaciones en lenguaje Python

UT2.- POO y Diseño modular

Tarea de Repaso

Diseña una aplicación Python que permita gestionar **cuentas bancarias** asociadas a diferentes tipos de productos (Cuentas de Ahorro, Cuentas Corrientes).

Todas las cuentas deben tener los siguientes **atributos privados**:

- **Número de Cuenta (IBAN)** (cadena, único e irrepetible)
- **Titular** (cadena)
- **Saldo** (flotante, valor inicial no negativo)
- **Fecha de Apertura** (cadena en formato DD-MM-AAAA). Consulta el módulo **datetime** de la librería estándar de Python.

Además, se deben desarrollar **métodos** para:

- **Realizar un Depósito** (aumentar el saldo).
- **Retornar el Saldo Actual** en formato de moneda (ej: "1.234,56 €").
- **Retornar una representación** de la cuenta con el formato: "IBAN: [Número de Cuenta] - Titular: [Titular] - Saldo: [Saldo Actual]".
- **Comprobar si dos cuentas tienen el mismo titular** sobreescribiendo el operador **!=**.

Restricción: No se podrán crear **instancias** de la clase directamente. Los métodos de **Depósito** y **Retorno de Saldo** serán de obligatoria implementación (o sobrescritura) en las subclases.

Lanza las **excepciones** para:

- si el saldo inicial es negativo
- o si el IBAN ya está registrado.

La cuentas bancarias pueden ser de dos tipos:

Cuentas de Ahorro

- **Atributo adicional privado: Tasa de Interés Anual** (flotante, valor positivo, e.g., 0.02 para 2%).
- **Métodos**
 - **Reintegro** (método a implementar) solo puede realizarse si el saldo resultante es mayor o igual a **100.00 €** (saldo mínimo).

- **Aplicar Interés** (Añade el interés anual simple al saldo actual, asumiendo un período de 1 año).

Cuenta Corriente

- **Atributo adicional privado: Límite de Descubierto** (flotante, valor positivo, e.g., 500.00 €).
- El **reintegro** (método a implementar) puede llevar el saldo a ser negativo, pero nunca debe superar el **Límite de Descubierto** (ej: si el límite es 500 €, el saldo mínimo es **-500 €**).
- **Excepción:** Lanzar una excepción y no permitir el retiro si la operación excede el límite de descubierto.

Crea un paquete para las clases anteriores y, a partir de este, crea un **programa de prueba** que valide:

- Creación de los diferentes tipos Cuentas Válidas
- Pruebas de Polimorfismo y Representación
- Pruebas los métodos de las cuenta de ahorro: depósitos, reintegros y aplicación de intereses
- Pruebas los métodos de las cuentas corrientes: depósitos y reintegros.